RGPVNOTES.IN

Subject Name: **Database Management System**

Subject Code: **CS-5003**

Semester: **5th**

LIKE & FOLLOW US ON FACEBOOK
facebook.com/rgpvnotes.in

Downloaded from be.rgpvnotes.in

Unit-III

Normalization:

Normalization is a database design technique which organizes tables in a manner that reduces redundancy and dependency of data. It divides larger tables to smaller tables and link them using relationships.

If a database design is not perfect, it may contain anomalies, which are like a bad dream for any database administrator. Managing a database with anomalies is next to impossible.

- **Update anomalies** – If data items are scattered and are not linked to each other properly, then it could lead to strange situations. For example, when we try to update one data item having its copies scattered over several places, a few instances get updated properly while a few others are left with old values. Such instances leave the database in an inconsistent state.
- **Deletion anomalies** – We tried to delete a record, but parts of it was left undeleted because of unawareness, the data is also saved somewhere else.
- **Insert anomalies** – We tried to insert data in a record that does not exist at all.

Normalization is a method to remove all these anomalies and bring the database to a consistent state.

Here are the most commonly used normal forms:

- First normal form(1NF)
- Second normal form(2NF)
- Third normal form(3NF)
- Boyce & Codd normal form (BCNF)

Normal forms:

First normal form (1NF)

As per the rule of first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic values.

Example: Suppose a company wants to store the names and contact details of its employees. It creates a table that looks like this:

emp_id	emp_name	emp_address	emp_mobile
101	Harish	New Delhi	8912312390
102	Jonny	Kanpur	8812121212 9900012222
103	Ronny	Chennai	7778881212
104	Lokesh	Bangalore	9990000123 8123450987

Two employees (Jon & Lester) are having two mobile numbers so the company stored them in the same field as you can see in the table above.

This table is **not in 1NF** as the rule says "each attribute of a table must have atomic (single) values", the emp_mobile values for employees Jon & Lester violates that rule.

To make the table complies with 1NF we should have the data like this:

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

emp_id	emp_name	emp_address	emp_mobile
101	Harish	New Delhi	8912312390
102	Jonny	Kanpur	8812121212
102	Jonny	Kanpur	9900012222
103	Ronny	Chennai	7778881212
104	Lokesh	Bangalore	9990000123
104	Lokesh	Bangalore	8123450987

Second normal form (2NF)

A table is said to be in 2NF if both the following conditions hold:

- Table is in 1NF (First normal form)
- No non-prime attribute is dependent on the proper subset of any candidate key of table.

An attribute that is not part of any candidate key is known as non-prime attribute.

Example: Suppose a school wants to store the data of teachers and the subjects they teach. They create a table that looks like this: Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

teacher_id	subject	teacher_age
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

Candidate Keys: {teacher_id,subject}

Non-prime attribute: teacher_age

The table is in 1 NF because each attribute has atomic values. However, it is not in 2NF because non-prime attribute teacher_age is dependent on teacher_id alone which is a proper subset of candidate key. This violates the rule for 2NF as the rule says "**no** non-prime attribute is dependent on the proper subset of any candidate key of the table".

To make the table complies with 2NF we can break it in two tables like this:

teacher_details table:

teacher_id	teacher_age
111	38
222	38
333	40

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

teacher_subject table:

teacher_id	subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Now the tables comply with Second normal form (2NF).

Third Normal form (3NF)

A table design is said to be in 3NF if both the following conditions hold:

- Table must be in 2NF
- **Transitive functional dependency** of non-prime attribute on any super key should be removed.

An attribute that is not part of any **candidate key** is known as non-prime attribute.

In other words, 3NF can be explained like this: A table is in 3NF if it is in 2NF and for each functional dependency X-> Y at least one of the following conditions hold:

- X is a **super key** of table
- Y is a prime attribute of table

An attribute that is a part of one of the candidate keys is known as prime attribute.

Example: Suppose a company wants to store the complete address of each employee, they create a table named employee_details that looks like this:

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

Super keys: {emp_id}, {emp_id, emp_name}, {emp_id, emp_name, emp_zip}…so on

Candidate Keys: {emp_id}

Non-prime attributes: all attributes except emp_id are non-prime as they are not part of any candidate keys.

Here, emp_state, emp_city & emp_district dependent on emp_zip. And, emp_zip is dependent on emp_id that makes non-prime attributes (emp_state, emp_city & emp_district) transitively dependent on super key (emp_id). This violates the rule of 3NF.

To make this table complies with 3NF we have to break the table into two tables to remove the transitive dependency:

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

Employee table:

emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

Employee_zip table:

emp_zip	emp_state	emp_city	emp_district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

Boyce Codd normal form (BCNF)

It is an advance version of 3NF that's why it is also referred as 3.5NF. BCNF is stricter than 3NF. A table complies with BCNF if it is in 3NF and for every **functional dependency** X->Y, X should be the super key of the table.

Example: Suppose there is a company wherein employees work in **more than one department**. They store the data like this:

emp_id	emp_nationality	emp_dept	dept_type	dept_no_of_emp
1001	Indian	Production and planning	D001	200
1001	Indian	stores	D001	250
1002	American	design and technical support	D134	100
1002	American	Purchasing department	D134	600

Functional dependencies in the table above:

emp_id -> emp_nationality

emp_dept -> {dept_type, dept_no_of_emp}

Candidate key: {emp_id, emp_dept}

The table is not in BCNF as neither emp_id nor emp_dept alone are keys.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

To make the table comply with BCNF we can break the table in three tables like this:

Emp_nationality table:

emp_id	emp_nationality
1001	Indian
1002	American

Emp_dept table:

emp_dept	dept_type	dept_no_of_emp
Production and planning	D001	200
Stores	D001	250
design and technical support	D134	100
Purchasing department	D134	600

Emp_dept_mapping table:

emp_id	emp_dept
1001	Production and planning
1001	stores
1002	design and technical support

Functional dependencies:
emp_id -> emp_nationality
emp_dept -> {dept_type, dept_no_of_emp}
Candidate keys:
For first table: emp_id
For second table: emp_dept
For third table: {emp_id, emp_dept}
This is now in BCNF as in both the functional dependencies left side part is a key.

Functional Dependency:
Functional dependency (FD) is a set of constraints between two attributes in a relation. Functional dependency says that if two tuples have same values for attributes A1, A2..., An, then those two tuples must have to have same values for attributes B1, B2, ..., Bn.
Functional dependency is represented by an arrow sign (→) that is, X→Y, where X functionally determines Y. The left-hand side attributes determine the values of attributes on the right-hand side.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

Fully Functional Dependency

A functional dependency P → Q is fully functional dependency if removal of any attribute A from P means that the dependency does not hold any more. or

n a relation R, an attribute Q is said to be fully functional dependent on attribute P, if it is functionally dependent on P and not functionally dependent on any proper subset of P. The dependency P → Q is left reduced, there being no extraneous attributes in the left-hand side of the dependency.

If AD → C, is fully functional dependency, then we cannot remove A or D. i.e. C is fully functional dependent on AD. If we are able to remove A or D, then it is not fully functional dependency.

EMPLOYEE

				FK
ENAME	SSN	BDATE	ADDREDD	DNUMBER

DEPARTMENT

		FK
DNAME	DNUMBER	DMGRESSN

DEPT_LOCATIONS

FK	
DNUMBER	DLOCATION

PK

PROJECT

			FK
PNAME	PNUMBER	PLOCATION	DNUM

PK

WORKS_ON

FK	FK	
SSN	PNUMBER	HOURS

{SSN, PNUMBER} → HOURS is a full FD since neither SSN → HOURS nor PNUMBER → HOURS hold

{SSN, PNUMBER} → ENAME is not a full FD (it is called a partial dependency) since SSN → ENAME also holds

Partial Functional Dependency –

A Functional Dependency in which one or more non key attributes are functionally depending on a part of the primary key is called partial functional dependency. or

where the determinant consists of key attributes, but not the entire primary key, and the determined consist of non-key attributes.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

For example, Consider a Relation R(A,B,C,D,E) having
FD : AB → CDE where PK is AB.

Then, { A → C; A → D; A → E; B → C; B → D; B → E }
all are Partial Dependencies.

Transitive Dependency –

Given a relation R(A,B,C) then dependency like A–>B, B–>C is a transitive dependency, since A–>C is implied .

In the above Fig,
SSN --> DMGRSSN is a transitive FD
{since SSN --> DNUMBER and DNUMBER --> DMGRSSN hold}

SSN --> ENAME is non-transitive FD since there is no set of attributes X
where SSN --> X and X --> ENAME.

Trivial Functional Dependency:

- **Trivial** – If a functional dependency (FD) X → Y holds, where Y is a subset of X, then it is called a trivial FD. Trivial FDs always hold.
- **Non-trivial** – If an FD X → Y holds, where Y is not a subset of X, then it is called a non-trivial FD.

FD Axioms

Understanding: Functional Dependencies are recognized by analysis of the real world; no automation or algorithm. Finding or recognizing them are the database designer's task.FD manipulations:

Soundness -- no incorrect FD's are generated

Completeness -- all FD's can be generated

Axiom Name	Axiom	Example
Reflexivity	if a is set of attributes, b ⊆ a, then a →b	*SSN,Name → SSN*
Augmentation	if a→ b holds and c is a set of attributes, then ca→cb	*SSN → Name* then *SSN,Phone → Name, Phone*
Transitivity	if a →b holds and b→c holds, then a→ c holds	*SSN →Zip* and *Zip → City* then *SSN →City*
Union or Additivity *	if a → b and a → c holds then a→ bc holds	*SSN→Name* and *SSN→Zip* then *SSN→Name,Zip*
Decomposition or Projectivity*	if a → bc holds then a → b and a → c holds	*SSN→Name,Zip* then *SSN→Name* and *SSN→Zip*
Pseudotransitivity*	if a → b and cb → d hold then ac → d holds	*Address → Project* and *Project,Date →Amount* then *Address,Date → Amount*

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

Decomposition:

Decomposition in DBMS is nothing but another name for Normalization. ... Normal forms in a database or the concept of Normalization makes a Relation or Table free from insert/update/delete anomalies and saves space by removing duplicate data.

- Decomposition is the process of breaking down in parts or elements.
- It replaces a relation with a collection of smaller relations.
- It breaks the table into multiple tables in a database.
- It should always be lossless, because it confirms that the information in the original relation can be accurately reconstructed based on the decomposed relations.
- If there is no proper decomposition of the relation, then it may lead to problems like loss of information.

Properties of Decomposition

1. Lossless Decomposition
2. Dependency Preservation
3. Lack of Data Redundancy

1. Lossless Decomposition

- Decomposition must be lossless. It means that the information should not get lost from the relation that is decomposed.
- It gives a guarantee that the join will result in the same relation as it was decomposed.

Example:

Let's take 'E' is the Relational Schema, With instance 'e'; is decomposed into: E1, E2, E3, ... En; with instance: e1, e2, e3, ... en, If e1 ⋈ e2 ⋈ e3 . . . . ⋈ en, then it is called as **'Lossless Join Decomposition'.**

- In the above example, it means that, if natural joins of all the decomposition give the original relation, then it is said to be lossless join decomposition.

Example: <Employee_Department> Table

Eid	Ename	Age	City	Salary	Deptid	DeptName
E001	ABC	29	Pune	20000	D001	Finance
E002	PQR	30	Pune	30000	D002	Production
E003	LMN	25	Mumbai	5000	D003	Sales
E004	XYZ	24	Mumbai	4000	D004	Marketing
E005	STU	32	Bangalore	25000	D005	Human Resource

- Decompose the above relation into two relations to check whether a decomposition is lossless or lossy.
- Now, we have decomposed the relation that is Employee and Department.

Relation 1 : <Employee> Table

Eid	Ename	Age	City	Salary

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

E001	ABC	29	Pune	20000
E002	PQR	30	Pune	30000
E003	LMN	25	Mumbai	5000
E004	XYZ	24	Mumbai	4000
E005	STU	32	Bangalore	25000

- Employee Schema contains (Eid, Ename, Age, City, Salary).

Relation 2: <Department> Table

Deptid	Eid	DeptName
D001	E001	Finance
D002	E002	Production
D003	E003	Sales
D004	E004	Marketing
D005	E005	Human Resource

- Department Schema contains (Deptid, Eid, DeptName).
- So, the above decomposition is a Lossless Join Decomposition, because the two relations contain one common field that is 'Eid' and therefore join is possible.
- Now apply natural join on the decomposed relations.

Employee ⋈ Department

Eid	Ename	Age	City	Salary	Deptid	DeptName
E001	ABC	29	Pune	20000	D001	Finance
E002	PQR	30	Pune	30000	D002	Production
E003	LMN	25	Mumbai	5000	D003	Sales
E004	XYZ	24	Mumbai	4000	D004	Marketing
E005	STU	32	Bangalore	25000	D005	Human Resource

Hence, the decomposition is Lossless Join Decomposition.

- If the <Employee> table contains (Eid, Ename, Age, City, Salary) and <Department> table contains (Deptid and DeptName), then it is not possible to join the two tables or relations, because there is no common column between them. And it becomes Lossy Join Decomposition.

2. Dependency Preservation

- Dependency is an important constraint on the database.
- Every dependency must be satisfied by at least one decomposed table.
- If {A → B} holds, then two sets are functional dependent. And, it becomes more useful for checking the dependency easily if both sets in a same relation.
- This decomposition property can only be done by maintaining the functional dependency.
- In this property, it allows to check the updates without computing the natural join of the database structure.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

3. Lack of Data Redundancy

- Lack of Data Redundancy is also known as a Repetition of Information.
- The proper decomposition should not suffer from any data redundancy.
- The careless decomposition may cause a problem with the data.
- The lack of data redundancy property may be achieved by Normalization process.

Null values and dangling tuples:

Null Values: The SQL NULL is the term used to represent a missing value. A NULL value in a table is a value in a field that appears to be blank. A field with a NULL value is a field with no value.

Dangling tuples: An attribute of the join operator is that it is possible for certain tuples to be "dangling"; that is, they fail to match any tuple of the other relation in the common attributes. Dangling tuples do not have any trace in the result of the join, so the join may not represent the data of the original relations completely.

Multivalued dependency:

Multivalued dependency occurs when there are more than one independent multivalued attributes in a table.
, a **multivalued dependency** is a full constraint between two sets of attributes in a relation.
In contrast to the functional dependency, the **multivalued dependency** requires that certain tuples be present in a relation. Therefore, a multivalued dependency is a special case of tuple-generating dependency. The multivalued dependency plays a role in the 4NF database normalization.
A multivalued dependency is a special case of a join dependency, with only two sets of values involved, i.e. it is a binary join dependency.
For example: Consider a bike manufacture company, which produces two colors (Black and white) in each model every year.

bike_model	manuf_year	color
M1001	2007	Black
M1001	2007	Red
M2012	2008	Black
M2012	2008	Red
M2222	2009	Black
M2222	2009	Red

Here columns manuf_year and color are independent of each other and dependent on bike_model. In this case these two columns are said to be multivalued dependent on bike_model. These dependencies can be represented like this:
bike_model ->> manuf_year
bike_model ->> color

Query optimization:

Query optimization is a function of many relational database management systems. The query optimizer attempts to determine the most efficient way to execute a given query by considering the possible query plans. Alternative ways of evaluating a given query

- Equivalent expressions
- Different algorithms for each operation

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

What is Query Optimization?

Suppose you were given a chance to visit 15 pre-selected different cities in Europe. The only constraint would be Time. Would you have a plan to visit the cities in any order? Place the 15 cities in different groups based on their proximity to each other. Start with one group and move on to the next group. Important point made over here is that you would have visited the cities in a more organized manner, and the 'Time' constraint mentioned earlier would have been dealt with efficiently.

Query Optimization works in a similar way: There can be many different ways to get an answer from a given query. The result would be same in all scenarios. DBMS strive to process the query in the most efficient way (in terms of 'Time') to produce the answer.

Steps in a Cost-based query optimization:

1. Parsing
2. Transformation
3. Implementation
4. Plan selection based on cost estimates

Query Flow:

- **Query Parser** – Verify validity of the SQL statement. Translate query into an internal structure using relational calculus.
- **Query Optimizer** – Find the best expression from various different algebraic expressions. Criteria used is 'Cheapness'
- **Code Generator/Interpreter** – Make calls for the Query processor as a result of the work done by the optimizer.
- **Query Processor** – Execute the calls obtained from the code generator.

There are two main techniques for implementing Query Optimization:

The **first** technique is based on ***Heuristic Rules*** for ordering the operations in a query execution strategy.

The **second** technique involves the ***systematic estimation*** of the cost of the different execution strategies and choosing the execution plan with the *lowest cost*.

- Semantic query optimization is used with the combination with the heuristic query transformation rules.
- It uses constraints specified on the database schema such as unique attributes and other more complex constraints, in order to modify one query into another query that is more efficient to execute.

1. Heuristic Rules: -

- The heuristic rules are used as an optimization technique to modify the internal representation of the query.
- Usually, heuristic rules are used in the form of query tree of query graph data structure, to improve its performance.
- One of the main heuristic rules is to apply SELECT operation before applying the JOIN or other BINARY operations.
- This is because the size of the file resulting from a binary operation such as JOIN is usually a multi-value function of the sizes of the input files.
- The SELECT and PROJECT reduced the size of the file and hence, should be applied before the JOIN or other binary operation.
- Heuristic query optimizer transforms the initial (canonical) query tree into final query tree using equivalence transformation rules.
- This final query tree is efficient to execute.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

For example, consider the following relations:

Employee (EName, EID, DOB, EAdd, Sex, ESalary, EDeptNo)

Department (DeptNo, DeptName, DeptMgrID, Mgr_S_date)

DeptLoc (DeptNo, Dept_Loc)

Project (ProjName, ProjNo, ProjLoc, ProjDeptNo)

WorksOn (E-ID, P-No, Hours)

Dependent (E-ID, DependName, Sex, DDOB, Relation)

- Now let us consider the query in the above database to find the name of employees born after 1970 who work on a project named 'Growth'.

SELECT EName

FROM Employee, WorksOn, Project

WHERE ProjName = 'Growth' AND ProjNo = P-No

AND EID = E-ID AND DOB > '31-12-1970';

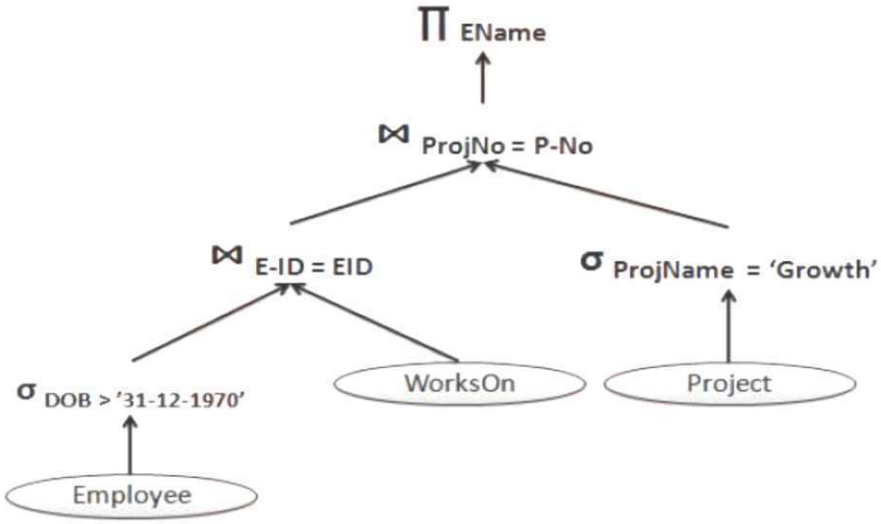

Fig 3.1

General Transformation Rules: -

Transformation rules are used by the query optimizer to transform one relational algebra expression into an equivalent expression that is more efficient to execute.

- A relation is considering as equivalent of another relation if two relations have the same set of attributes in a different order but representing the same information.
- These transformation rules are used to restructure the initial (canonical) relational algebra query tree attributes during query decomposition.

1. Cascade of σ :-

σ c1 AND c2 AND …AND cn (R) = σ c1 (σ c2 (…(σ cn (R))…))

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

2. Commutativity of σ :-

σ C1 (σ C2 (R)) = σ C2 (σ C1 (R))

3. Cascade of Л :-

Л List1 (Л List2 (…(Л List n (R))…)) = Л List1 (R)

4. Commuting σ with Л :-

Л A1,A2,A3…An (σ C (R)) = σ C (Л A1,A2,A3…An (R))

5. Commutativity of ⋈ AND x :-

R ⋈ c S = S ⋈ c R

R x S = S x R

6. Commuting σ with ⋈ or x :-

If all attributes in selection condition c involved only attributes of one of the relation schemas (R).

σ c (R ⋈ S) = (σ c (R)) ⋈ S

Alternatively, selection condition c can be written as (c1 AND c2) where condition c1 involves only attributes of R and condition c2 involves only attributes of S then:

σ c (R ⋈ S) = (σ c1 (R)) ⋈ (σ c2 (S))

7. Commuting Л with ⋈ or x :-

- he projection list L = {A1,A2,..An,B1,B2,…Bm}.

-A1…An attribute of R and B1…Bm attributes of S.

-Join condition C involves only attributes in L then :

ЛL (R ⋈ c S) = (ЛA1,…An (R)) ⋈c (ЛB1,…Bm(S))

8. Commutativity of SET Operation: -

- R ∪ S = S ∪ R

- R ∩ S = S ∩ R

Minus (R-S) is not commutative.

9. Associatively of ⋈, x, ∩, and ∪ :-

- If Ø stands for any one of these operation throughout the expression then :

(R Ø S) Ø T = R Ø (S Ø T)

10. Commutativity of σ with SET Operation: -

- If Ø stands for any one of three operations (∪,∩,and-) then :

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

σ c (R Ø S) = (σ c (R)) ∪ (σ c (S))

Л c (R Ø S) = (Л c (R)) ∪ (Лc (S))

11. The Л operation comute with ∪ :-

Л L (R ∪ S) = (Л L(R)) ∪ (Л L(S))

12. Converting a (σ,x) sequence with ∪

(σ c (R x S)) = (R ⋈ c S)

Heuristic Optimization Algorithm: -

- The Database Management System Use Heuristic Optimization Algorithm that utilizes some of the transformation rules to transform an initial query tree into an optimized and efficient executable query tree.
- The steps of the heuristic optimization algorithm that could be applied during query processing and optimization are:

  Step-1: -
- Perform SELECT operation to reduce the subsequent processing of the relation:
- Use the transformation rule 1 to break up any SELECT operation with conjunctive condition into a cascade of SELECT operation.

  Step-2: -
- Perform commutativity of SELECT operation with other operation at the earliest to move each SELECT operation down to query tree.
- Use transformation rules 2, 4, 6 and 10 concerning the commutativity of SELECT with other operation such as unary and binary operations and move each select operation as far down the tree as is permitted by the attributes involved in the SELECT condition. Keep selection predicates on the same relation together.

  Step-3: -
- Combine the Cartesian product with subsequent SELECT operation whose predicates represents the join condition into a JOIN operation.
- Use the transformation rule 12 to combine the Cartesian product operation with subsequent SELECT operation.

  Step-4: -
- Use the commutativity and associativity of the binary operations.
- Use transformation rules 5 and 9 concerning commutativity and associativity to rearrange the leaf nodes of the tree so that the leaf node with the most restrictive selection operation is executed first in the query tree representation. The most restrictive SELECT operation means:
- Either the one that produce a relation with a fewest tuples or with smallest size.
- The one with the smallest selectivity.

  Step-5: -

  Perform the projection operation as early as possible to reduce the cardinality of the relation and the subsequent processing of the relation, and move the Projection operations as far down the query tree as possible.

  o Use transformation rules 3, 4, 7 and 10 concerning the cascading and commuting of projection operations with other binary operation. Break down and move the projection attributes down the tree as far as needed. Keep the projection attributes in the same relation together.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

Step-6: -

- Compute common expression once.
- Identify sub-tree that represent group of operations that can be executed by a single algorithm.

Implementing the SELECT Operation

Simple selection
S1: Linear search
S2: Binary search
S3a: Primary index
S3b: Hash key
S4: Primary index, multiple records
(For a comparative clause like >, ≤) Use the index to retrieve the bounding = condition, then iterate.
S5: Clustering index, multiple records
For an equality clause that's nonkey.
S6: Secondary index (B^+-Tree)
Complex selection
A conjunctive selection combines subclauses with logical And.
S7: Conjunctive selection, individual index
Retrieve records based on the indexed attribute that satisfy its associated condition. Check each one whether it satisfies the remaining conditions.
S8: Conjunctive selection, composite index
If the attributes participating the composite index have equality conditions, use it directly.
S9: Conjunctive selection, intersection of result sets
Retrieve records satisfying the clause of their indexed attributes, separately, to form result sets. Compute the intersection of these sets.

Condition selectivity
The *selectivity* sl is the ratio of tuples satisfying the selection condition to the total tuples in the relation. Low values of sl are usually desirable. More importantly, a DBMS will tend to keep an estimate of the distribution of values among the attributes of the rows of a table, as a *histogram*, to be able to estimate the selectivity of query operations.

Disjunctive selection conditions
These have the form $\sigma_{k1 \vee k2 \vee \ldots}$(Relation). About the only hope to optimize is to use a separate index on each sub condition k_i and compute the union of their results.

PROJECT operation

If the < attribute list > of the PROJECT operation π (R) includes a key of R, then the number of tuples in the projection result is equal to the number of tuples in R, but only with the values for the attributes in < attribute list > in each tuple.

If the < attribute list > does not contain a key of R, duplicate tuples must be eliminated. The following methods can be used to eliminate duplication.

- Sorting the result of the operation and then eliminating duplicate tuples.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

- Hashing the result of the operation into hash file in memory and check each hashed record against those in the same bucket; if it is a duplicate, it is not inserted.

CARTESIAN PRODUCT operation: – It is an extremely expensive operation. Hence, it is important to avoid this operation and to substitute other equivalent operations during optimization.

Translating SQL Queries into Relation Algebra

An SQL query is first translated into an equivalent extended relation algebra expression (as a query tree) that is then optimized.

Query block in SQL: the basic unit that can be translated into the algebraic operators and optimized. A query block contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clauses.

Consider the following SQL query.

SELECT LNAME, FNAME FROM EMPLOYEE WHERE SALARY > (SELECT MAX (SALARY) FROM EMPLOYEE WHERE DNO=5);

We can decompose it into two blocks: Inner block: Outer block:

(SELECT MAX (SALARY) SELECT LNAME, FNAME FROM EMPLOYEE FROM EMPLOYEE WHERE DNO=5) WHERE SALARY > c

Then translate to algebra expressions: ∗ Inner block: =

MAX SALARY (σDNO=5 EMPLOY EE) ∗ Outer block: πLNAME, F NAME(σSALARY >c EMPLOY EE)

Optimization methods:

Optimization: Heuristical Processing Strategies

- Perform selection operations as early as possible.
- Keep predicates on same relation together.
- Combine Cartesian product with subsequent selection whose predicate represents join condition into a join operation.
- Use associativity of binary operations to rearrange leaf nodes so leaf nodes with most restrictive selection operations executed first.
- Perform projection as early as possible**.**
- Keep projection attributes on same relation together.
- Compute common expressions once.
  - If common expression appears more than once, and result not too large, store result and reuse it when required.
  - Useful when querying views, as same expression is used to construct view each time.

One of the main heuristic rules is to apply SELECT and PROJECT operations *before* applying the JOIN or other binary operations. This is because the size of the file resulting from a binary operation—such as JOIN—is usually a multiplicative function of the sizes of the input files. The SELECT and PROJECT operations reduce the size of a file and hence should be applied *before* a join or other binary operation.

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
  - Perform selection early (reduces the number of tuples)
  - Perform projection early (reduces the number of attributes)

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

- Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.

Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Pcustomer_name((sbranch_city = “Brooklyn” (branch) account) depositor)

1) When we compute (sbranch_city = “Brooklyn” (branch) account)
we obtain a relation whose schema is:
(branch_name, branch_city, assets, account_number, balance)
2) Push projections using equivalence rules; eliminate unneeded attributes from intermediate results to get: Pcustomer_name ((Paccount_number ((sbranch_city = “Brooklyn” (branch) account)) depositor)
3) Performing the projection as early as possible reduces the size of the relation to be joined.

Optimization: Cost Estimation for RA Operations:

- Many different ways of implementing RA operations.
- Aim of QO is to choose most efficient one.
- Use formulae that estimate costs for a number of options, and select one with lowest cost.
- Consider only cost of disk access, which is usually dominant cost in QP.
- Many estimates are based on cardinality of the relation, so need to be able to estimate this.

 Follow us on facebook to get real-time updates from RGPV

Downloaded from be.rgpvnotes.in

Notes

Course: BE

College: CDGI

Year/Semester: 3rd Year / 5th & 6th Sem

Subject Code: CS-5003

Subject: DBMS

CONNECT WITH US

LIKE & FOLLOW US ON FACEBOOK
facebook.com/rgpv.tk

FOLLOW US ON GOOGLE+
www.goo.gl/fVRsQ1

SUBSCRIBE US ON YOUTUBE
www.goo.gl/T3dQzX

WWW.RGPV.TK

Follow us on facebook to get real-time updates from RGPV

RGPVNOTES.IN

We hope you find these notes useful.

You can get previous year question papers at
https://qp.rgpvnotes.in .

If you have any queries or you want to submit your study notes please write us at
rgpvnotes.in@gmail.com

LIKE & FOLLOW US ON FACEBOOK
facebook.com/rgpvnotes.in